6.7 Record Types

- *Record*, tek tek elemanların names'lerle tanımlandığı, muhtemelen heterojen bir veri elemanları topluluğudur.
- Tasarım sorunları:
  - Alana yapılan referansların sözdizimsel biçimi nedir?
  - Eliptik referanslara izin verilir mi?

Copyright © 2015 Pearson. All rights reserved.

6.7.1 COBOL'da Records Tanımı

- COBOL, iç içe geçmiş kayıtları göstermek için seviye numaraları kullanır; diğerleri ise özyinelemeli tanımlama kullanır.

```
01 EMP-REC.
   02 EMP-NAME.
      05 FIRST PIC X(20).
      05 MID   PIC X(10).
      05 LAST  PIC X(20).
   02 HOURLY-RATE PIC 99V99.
```

Copyright © 2015 Pearson. All rights reserved.

6.7.2 References to Records

- Record alanı referansları
  1. COBOL
  field_name `OF` record_name_1 `OF` ... `OF` record_name_n
  2. Diğerleri (nokta notasyonu)
  record_name_1.record_name_2. ... record_name_n.field_name

- Tam nitelikli referanslar tüm record names'leri içermelidir.

- Eliptik referanslar, referansın anlamı açık olduğu sürece record names'lerin çıkarılmasına izin verir. Örneğin COBOL'da `FIRST, FIRST OF EMP-NAME`, ve `FIRST` of EMP-REC, çalışanın ilk adının eliptik referanslarıdır.

Copyright © 2015 Pearson. All rights reserved.

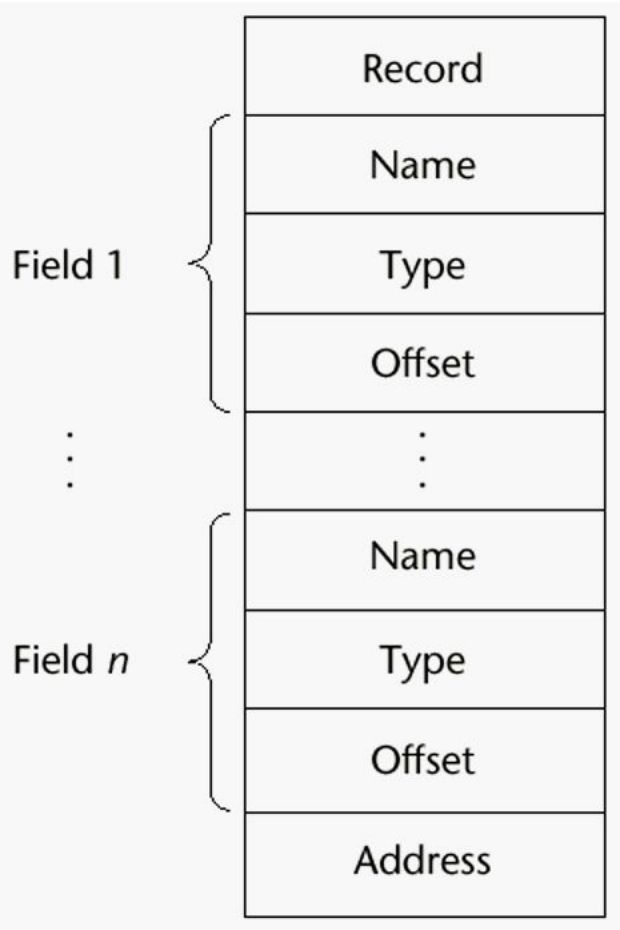
6.7.3 Değerlendirme ve Arrays'le Karşılaştırma

- Records, veri değerlerinin toplanması heterojen olduğunda kullanılır.
- Array elemanlarına erişim, record fields'a erişimden çok daha yavaştır, çünkü subscripts dinamiktir (field names statiktir).
- Dinamik subscripts record field erişiminde kullanılabilir, ancak bu durumda tip kontrolü yapılamaz ve işlem çok daha yavaş olur.

Copyright © 2015 Pearson. All rights reserved.

6.7.4 Implementation of Record Type

Records başlangıcına göre offset adresi her field ile ilişkilendirilir.



Copyright © 2015 Pearson. All rights reserved.

6.8 Tuple Types

- Tuple, elemanların isimlendirilmemesi dışında record'a benzer bir veri türüdür.
- Python, ML ve F#'da fonksiyonların birden fazla değer döndürmesini sağlamak için kullanılır.
  - Python
    - Lists ile yakından ilişkili, ancak değiştirilemez (immutable).
    - Tuple literal ile oluşturma

      ```
      myTuple = (3, 5.8, 'apple')
      ```

      Subscripts ile referanslandırılır (`1`'den başlar)

    Catenation with `+` ile birleştirilir (catenation) **`del`** ile silinir.

Copyright © 2015 Pearson. All rights reserved.

6.8 Tuple Types (Devam Ediyor)

- ML

```
val myTuple = (3, 5.8, 'apple');
```

  - Şu şekilde erişin:

    `#1(myTuple)` ilk eleman

  - Yeni bir tuple türü tanımlanabilir

```
type intReal = int * real;
```

- F#

```
let tup = (3, 5, 7)
```

```
let a, b, c = tup
```

Bu, bir tuple'ı bir tuple desenine atar. `(a, b, c)`

Copyright © 2015 Pearson. All rights reserved.

6.9 List Types

- Lisp ve Scheme'deki lists parantezlerle sınırlandırılır ve virgül kullanılmaz.

  `(A B C D)` and `(A (B C) D)`

- Veriler ve kodlar aynı formata sahiptir.

  Veri olarak, `(A B C)` tam anlamıyla neyse odur.

  Kod olarak, `(A B C)` parametreleri `B` ve `C`'ye uygulanan `A` fonksiyonudur.

- Interpreter, listenin hangisi olduğunu bilmelidir, bu nedenle veri ise, onu apostrof ile alıntı yaparız.

  `'(A B C)` is data

Copyright © 2015 Pearson. All rights reserved.

6.9 List Types (Devam Ediyor)

- Scheme'de List İşlemleri
  - `CAR`, list parametresinin ilk elemanını döndürür.

    `(CAR '(A B C))` returns `A`
  - `CDR`, ilk eleman kaldırıldıktan sonra liste parametresinin geri kalanını döndürür.

    `(CDR '(A B C))` returns `(B C)`
  - `CONS`, ilk parametresini ikinci parametresine, bir listeye koyarak yeni bir liste oluşturur.

    `(CONS 'A (B C))` returns `(A B C)`
  - `LIST`, parametrelerinin yeni bir listesini döndürür.

    `(LIST 'A 'B '(C D))` returns `(A B (C D))`

Copyright © 2015 Pearson. All rights reserved.

6.9 List Types (Devam Ediyor)

- ML'de List İşlemleri
  - Listeler parantez içinde yazılır ve elemanlar virgülle ayrılır.
  - Liste elemanları aynı türde olmalıdır
  - Scheme `CONS` fonksiyonu, ML'de bir ikili işlemcidir `::`

    `3 :: [5, 7, 9]` evaluates to `[3, 5, 7, 9]`
  - Scheme'da `CAR` ve `CDR` fonksiyonları sırasıyla `hd` ve `tl` olarak adlandırılmıştır.

Copyright © 2015 Pearson. All rights reserved.

6.9 List Types (Devam Ediyor)

- F# Lists
  - ML'deki gibi, ancak elemanlar noktalı virgülle ayrılır ve `hd` ile `tl List` sınıfının metotlarıdır.
- Python Lists
  - Liste veri türü, Python'un dizileri olarak da işlev görür.
  - Scheme, Common Lisp, ML ve F#'dan farklı olarak, Python'un listeleri değiştirilebilir.
  - Elemanlar herhangi bir türde olabilir
  - Atama içeren bir liste oluşturma

```
myList = [3, 5.8, "grape"]
```

Copyright © 2015 Pearson. All rights reserved.

6.9 List Types (Devam Ediyor)

- Python Lists (Devam Ediyor)
  - Liste elemanları, sıfırdan başlayan indekslerle subscripting ile referanslanır.

    `x = myList[1]` Sets `x` to `5.8`
  - Liste elemanları `del` ile silinebilir.

    ```
    del myList[1]
    ```
  - Liste Anlamaları – küme notasyonundan türetilmiştir

    ```
    [x * x for x in range(6) if x % 3 == 0]
    ```

    `range(12)` creates `[0, 1, 2, 3, 4, 5, 6]`

    Constructed list: `[0, 9, 36]`

Copyright © 2015 Pearson. All rights reserved.

6.9 List Types (Devam Ediyor)

- Haskell'ın Liste Anlamaları
  - Orijinal

```
[n * n | n <- [1..10]]
```

- F#'s Liste Anlamaları

```
let myArray = [|for i in 1 .. 5 -> [i * i) |]
```

- Hem C# hem de Java, sırasıyla `List` ve `ArrayList` adlı heap-dynamic collection class'ları aracılığıyla listeleri destekler.

Copyright © 2015 Pearson. All rights reserved.

6.10 Unions Types

- *Union*, değişkenlerinin yürütme sırasında farklı zamanlarda farklı türde değerleri depolamasına izin verilen bir türdür.
- Tasarım sorunu
  - Tip kontrolü gerekli midir?

Copyright © 2015 Pearson. All rights reserved.

6.10.1 Discriminated vs. Free Unions

- C ve C++, tür denetimi için dil desteği olmayan union yapıları sağlar; bu dillerdeki union, *free union* olarak adlandırılır.
- unions tür denetimi, her union'un *discriminant* adı verilen bir tür göstergesi içermesini gerektirir.
  - ML, Haskell ve F# tarafından desteklenir

Copyright © 2015 Pearson. All rights reserved.

6.10.2 Unions in F#

- OR kullanılarak bir tür ifadesiyle tanımlanmıştır.

```
type intReal =
    | IntValue of int
    | RealValue of float;;
```

`intReal` yeni türdür

`IntValue` ve `RealValue` constructors'dır.

`intReal` türünde bir değer oluşturmak için:

```
let ir1 = IntValue 17;;
let ir2 = RealValue 3.4;;
```

Copyright © 2015 Pearson. All rights reserved.

6.10.2 Unions in F# (Devam Ediyor)

Bir union'ın değerine erişim, desen eşleştirme ile yapılır.

```
match pattern with
    | expression_list1 -> expression1
    | ...
    | expression_listn -> expressionn
```

- Desen herhangi bir veri türü olabilir
- İfade listesi joker karakterler (_) içerebilir.

Copyright © 2015 Pearson. All rights reserved.

6.10.2 Unions in F# (Devam Ediyor)

Örneğin:

```
let a = 7;;
let b = "grape";;
let x = match (a, b) with
      | 4, "apple" -> apple
      | _, "grape" -> grape
      | _ -> fruit;;
```

Copyright © 2015 Pearson. All rights reserved.

6.10.2 Unions in F# (Devam Ediyor)

`intReal` union türünü görüntülemek için:

```
let printType value =
  match value with
      | IntVale value -> printfn "int"
      | RealValue value -> printfn "float";;
```

`ir1` ve `ir2` önceden tanımlanmışsa,

`printType ir1` returns `int`

`printType ir2` returns `float`

Copyright © 2015 Pearson. All rights reserved.

6.10.3 Unions'ın Değerlendirmesi

- Free unions güvenli değildir
  - Tip kontrolüne izin vermez
- Java ve C# unions'ı desteklemez.
  - Programlama dilinde güvenlik konusundaki artan endişeleri yansıtmaktadır.

Copyright © 2015 Pearson. All rights reserved.

6.11 Pointer ve Reference Types

- Bir *pointer* tipi değişken, bellek adreslerinden ve özel bir değer olan *nil*'den oluşan bir değer aralığına sahiptir.
- Dolaylı adresleme gücünü sağlar.
- Dinamik belleği yönetmek için bir yol sağlar.
- Bir pointer, depolamanın dinamik olarak oluşturulduğu alandaki bir konuma erişmek için kullanılabilir (genellikle *yığın (heap)* olarak adlandırılır).

Copyright © 2015 Pearson. All rights reserved.

6.11.1 Pointers Tasarım Sorunları

- Bir pointer değişkeninin scope'u ve lifetime'ı nedir?
- Heap-dynamic değişkeninin lifetime'ı nedir?
- Pointerlar, işaret edebilecekleri değer türleri açısından kısıtlı mıdır?
- Pointer'lar dinamik depolama yönetimi, dolaylı adresleme veya her ikisi için mi kullanılır?
- Dil, pointer türlerini, referans türlerini veya her ikisini birden desteklemeli mi?

Copyright © 2015 Pearson. All rights reserved.

6.11.2 Pointer Operations

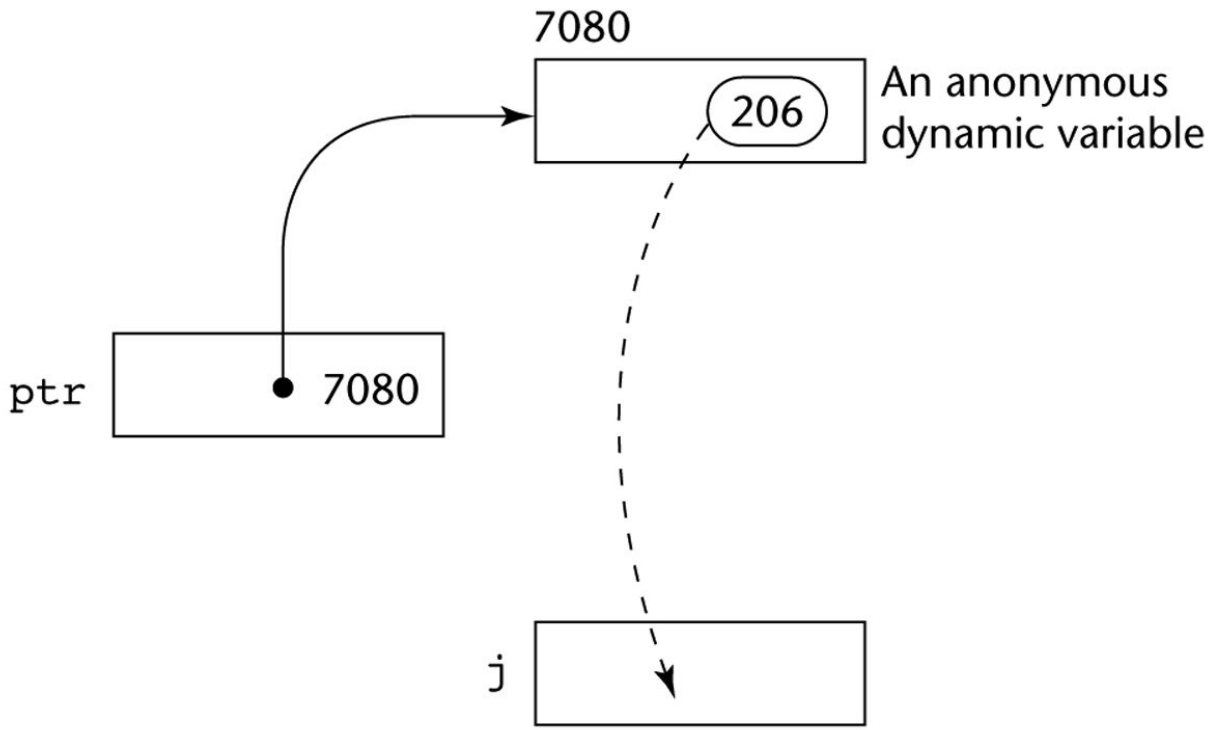
- İki temel işlem: atama ve referans kaldırma
- Atama, bir pointer değişkeninin değerini yararlı bir adrese ayarlamak için kullanılır.
- Dereferencing, pointer'ın değeriyle temsil edilen konumda depolanan değeri verir.
  - Dereferencing explicit veya implicit olabilir.
  - C++, * aracılığıyla explicit bir operasyon kullanır.

    `j = *ptr`

    `j`'yi `ptr`'de bulunan değere ayarlar

Copyright © 2015 Pearson. All rights reserved.

6.11.3 Pointer Assignment Illustrated



The assignment operation j = *ptr

Copyright © 2015 Pearson. All rights reserved.

6.11.4 Pointerlarla İlgili Sorunlar

- Sarkan pointer'lar (tehlikeli)
  - Bir pointer, ayrılmış bir heap-dinamik değişkeni işaret eder.
- Kayıp heap-dinamik değişken
  - Kullanıcı programı tarafından artık erişilemeyen, ayrılmış bir heap-dinamik değişken (genellikle çöp olarak adlandırılır)
    - Pointer `p1`, yeni oluşturulan bir heap-dinamik değişkeni işaret edecek şekilde ayarlanır.
    - Pointer `p1` daha sonra yeni oluşturulan başka bir heap-dinamik değişkeni işaret edecek şekilde ayarlanır.
    - Heap dinamik değişkenlerin kaybolması süreci, *bellek sızıntısı (memory leakage)* olarak adlandırılır.

Copyright © 2015 Pearson. All rights reserved.

6.11.5 C ve C++'da Pointers

- Son derece esnek ancak dikkatli kullanılmalıdır
- Pointerlar, ne zaman ve nerede ayrılmış olursa olsun herhangi bir değişkeni işaret edebilir.
- Dinamik depolama yönetimi ve adresleme için kullanılır.
- Pointer aritmetiği mümkündür
- Explicit dereferencing ve adres operatörleri
- Domain türü sabit olmak zorunda değildir (`void *`)

  `void *` herhangi bir türü işaret edebilir ve tür denetimi yapılabilir (referansı kaldırılamaz)

Copyright © 2015 Pearson. All rights reserved.

6.11.6 C ve C++'da Pointer Arithmetic

```
float stuff[100];
float *p;
p = stuff;
```

`*(p+5)`, `stuff[5]` ve `p[5]` eşdeğerdir.

`*(p+i)`, `stuff[i]` ve `p[i]` eşdeğerdir.

Copyright © 2015 Pearson. All rights reserved.

6.11.7 Reference Types

- C++, öncelikle biçimsel parametreler için kullanılan *referans türü (reference type)* adı verilen özel bir pointer türü içerir.
  - pass-by-reference ve pass-by-value avantajları
- Java, C++'ın referans değişkenlerini genişletir ve bunların pointer'ları tamamen değiştirmelerine izin verir.
  - Referanslar, adreslerden ziyade nesnelere yapılan referanslardır.
- C#, hem Java referanslarını hem de C++ pointer'larını içerir.

Copyright © 2015 Pearson. All rights reserved.

6.11.8 Pointers Değerlendirilmesi

- Sarkan pointer'lar ve sarkan nesneler, heap yönetimi gibi sorunlardır.
- Pointers, goto'lar gibidir; bir değişken tarafından erişilebilen hücrelerin aralığını genişletirler.
- Dinamik veri yapıları için pointerlar veya referanslar gereklidir; bu yüzden bunlar olmadan bir dil tasarlayamayız.

Copyright © 2015 Pearson. All rights reserved.

6.11.9 Representations of Pointers

- Büyük bilgisayarlar tekli değerler kullanır
- Intel mikroişlemciler segment ve offset kullanır

Copyright © 2015 Pearson. All rights reserved.

6.11.10 Dangling Pointer Problem

- *Tombstone*: heap-dinamik değişkenine işaret eden ekstra heap hücresi
  - Gerçek pointer değişkeni sadece tombstones gösterir.
  - Heap-dinamik değişken tahsis edildiğinde (de-allocated), tombstone kalır ancak nil olarak ayarlanır.
  - Zaman ve alan açısından maliyetli
- *Locks-and-keys*: Pointer değerleri (key, address) çiftleri olarak temsil edilir.
  - Heap-dinamik değişkenler, integer kilit değeri için değişken artı hücre olarak temsil edilir.
  - Heap-dinamik değişken ayrıldığında (allocated), lock değeri oluşturulur ve pointer'ın lock hücresine ve key hücresine yerleştirilir.

Copyright © 2015 Pearson. All rights reserved.

6.11.11 Heap Management

- Çok karmaşık bir çalışma zamanı süreci
- Single-size cells vs. variable-size cells
- Garbage geri kazanımı için iki yaklaşım
  - Reference counters (*istekli yaklaşım*): geri kazanım kademeli olarak gerçekleşir
  - Mark-sweep (*tembel yaklaşım*): değişken alanı listesi boşaldığında geri kazanım gerçekleşir

Copyright © 2015 Pearson. All rights reserved.

6.11.12 Reference Counter

- Reference counters: her hücrede, o anda hücreyi işaret eden pointerların sayısını depolayan bir sayaç bulundurmak
  - *Dezavantajlar*: gerekli alan, gerekli yürütme süresi, dairesel olarak bağlanan hücreler için komplikasyonlar
  - *Avantajlar*: doğası gereği artımlıdır, bu nedenle uygulama yürütmede önemli gecikmeler önlenir

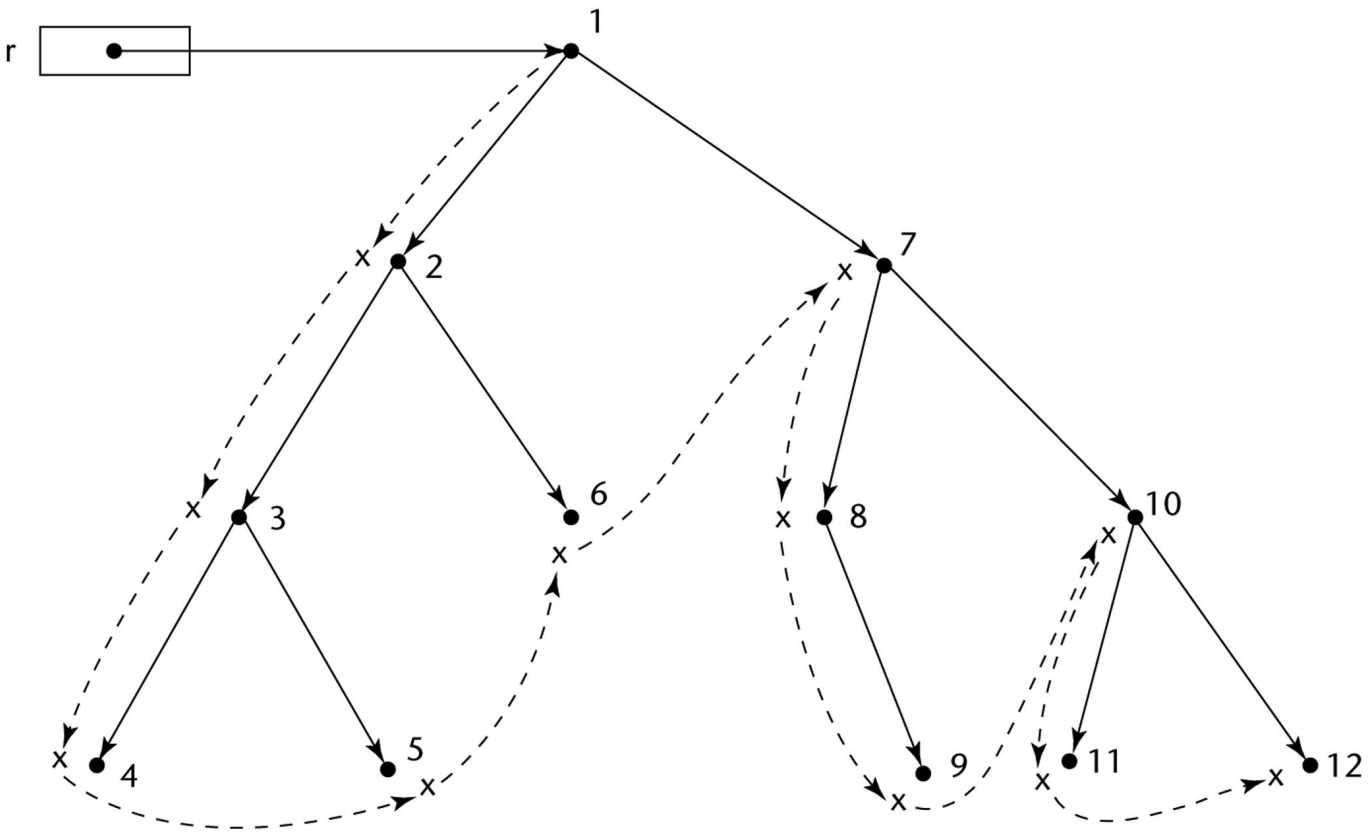
Copyright © 2015 Pearson. All rights reserved.

6.11.13 Mark-Sweep

- Çalışma zamanı sistemi, talep edildiği şekilde depolama hücrelerini tahsis eder (allocate) ve gerektiğinde hücrelerden pointer'ları ayırır; ardından mark-sweep işlemi başlar.
  - Her heap hücresi, toplama algoritması tarafından kullanılan ekstra bir bite sahiptir.
  - Tüm hücreler başlangıçta garbage olarak ayarlanır
  - Tüm pointer'lar heap'e izlenir ve ulaşılabilir hücreler garbage olarak işaretlenmez.
  - Tüm garbage hücreleri kullanılabilir hücreler listesine geri döndürülür
  - Dezavantajlar: orijinal haliyle, çok seyrek yapılıyordu. Yapıldığında, uygulama yürütmede önemli gecikmelere neden oluyordu. Günümüzün mark-sweep algoritmaları, bunu daha sık yaparak bu sorunu önlüyorlar — buna artımlı mark-sweep deniyor.

Copyright © 2015 Pearson. All rights reserved.

6.11.14 Marking Algorithm



Dashed lines show the order of node_marking

Copyright © 2015 Pearson. All rights reserved.

6.11.15 Variable-Size Cells

- Tek boyutlu hücrelerin tüm zorlukları ve daha fazlası
- Çoğu programlama dilinde gereklidir
- Mark-sweep kullanılırsa, ek sorunlar ortaya çıkar.
  - Heap içindeki tüm hücrelerin göstergelerinin ilk ayarı zordur.
  - Önemsiz olmayan marking süreci
  - Kullanılabilir alan listesini tutmak da ek yükün bir başka kaynağıdır.

Copyright © 2015 Pearson. All rights reserved.

6.12 Type Checking

- Operand ve operatör kavramlarını, alt programları ve atamaları da içerecek şekilde genelleştirmektir.
- *Type checking*, bir operatörün işlenenlerinin uyumlu tiplerde olmasını sağlama faaliyetidir.
- *Compatible type*, operatör için geçerli olan veya dil kuralları uyarınca derleyici tarafından üretilen kodla geçerli bir tipe örtük olarak dönüştürülebilen tiptir.
  - Bu otomatik dönüştürme, *coercion* olarak adlandırılır.
- Tip hatası, uygun olmayan tipteki bir operand'a bir operatörün uygulanmasıdır.

Copyright © 2015 Pearson. All rights reserved.

6.12 Type Checking (Devam Ediyor)

- Tüm type binding'ler statik ise, neredeyse tüm type checking'ler statik olabilir.
- Type binding'ler dinamik ise, type checking de dinamik olmalıdır.
- Bir programlama dili, tür hataları her zaman tespit edilirse güçlü bir şekilde türlenmiştir (*strongly typed*).
- Strong Typing'in Avantajı: tip hatalarına neden olan değişkenlerin yanlış kullanımlarının tespit edilmesini sağlar

Copyright © 2015 Pearson. All rights reserved.

6.13 Strong Typing

Dil örnekleri:

- C ve C++ böyle değildir: parameter type checking önlenebilir; unions type check'ten geçmez.
- Java ve C#, neredeyse (explicit type casting nedeniyle).

- ML ve F# böyledir.

Copyright © 2015 Pearson. All rights reserved.

6.13 Strong Typing (Devam Ediyor)

- Coercion kuralları strong typing'i büyük ölçüde etkiler; onu önemli ölçüde zayıflatabilirler (C++ ile ML ve F# karşılaştırması).

- Java, C++'ın strong typing'inin sadece yarısına sahip olsa da, 'Strong typing' özelliği hala Ada'nınkinden çok daha az etkilidir.

Copyright © 2015 Pearson. All rights reserved.

6.14 Name Type Equivalence

- *Name type equivalence*, iki değişkenin aynı bildirimde veya aynı tür adını kullanan bildirimlerde yer alması durumunda eşdeğer türlere sahip olduğu anlamına gelir.
- Uygulaması kolay ancak son derece kısıtlayıcı:
  - İnteger türlerinin alt aralıkları, integer türleriyle eşdeğer değildir.
  - Formal parametreler, karşılık gelen gerçek parametrelerle aynı türde olmalıdır.

Copyright © 2015 Pearson. All rights reserved.

6.15 Structure Type Equivalence

- *Structure type equivalence*, iki değişkenin türleri aynı yapıya sahipse, bu değişkenlerin türlerinin eşdeğer olduğu anlamına gelir.
- Daha esnek, ancak uygulaması daha zor

Copyright © 2015 Pearson. All rights reserved.

6.16 Type Equivalence (Devam Ediyor)

- İki structured types sorununu ele alalım:
  - Yapısal olarak aynı olan ancak farklı alan adları kullanan iki record türü eşdeğer midir?
  - İki array türü, subscriptleri farklı olması dışında aynıysa eşdeğer midir?

    (Örneğin `[1..10]` ve `[0..9]`)
  - Componentleri farklı yazılmışsa iki enumeration türü eşdeğer midir?
  - Structural type equivalence ile, aynı yapıdaki tipler arasında ayrım yapamazsınız (örneğin, farklı hız birimleri, her ikisi de float).

Copyright © 2015 Pearson. All rights reserved.

6.17 Theory and Data Types

- Type theory, matematik, mantık, bilgisayar bilimi ve felsefe alanlarında geniş bir çalışma alanıdır.
- Bilgisayar bilimlerinde yype theory iki dalı:
  - Practical – ticari dillerdeki veri türleri
  - Abstract – tipik lambda kalkulüsü

- Type system, bir dizi tip ve bunların programlarda kullanımını düzenleyen kurallardır.

Copyright © 2015 Pearson. All rights reserved.

6.17 Theory and Data Types (Devam Ediyor)

- Type System formal modeli, tip kurallarını tanımlayan bir dizi tip ve bir fonksiyonlar koleksiyonudur.
  - Fonksiyonlar için attribute grammar veya type map kullanılabilir.
  - Finite mappings – arrays ve fonksiyonları modeller.
  - Cartesian products – tuple'ları ve kayıtları modeller.
  - Set unions – union türleri modeller.
  - Subsets – subtypes modeller.

Copyright © 2015 Pearson. All rights reserved.

Özet

- Bir dilin veri türleri, o dilin stilini ve kullanışlılığını belirleyen en önemli unsurlardan biridir.
- Çoğu imperatif dilin temel veri türleri arasında numeric, character ve Boolean türleri bulunur.
- Kullanıcı tanımlı enumeration ve alt aralık türleri kullanışlıdır ve programların okunabilirliğini ve güvenilirliğini artırır.
- Array'ler ve records çoğu dilde mevcuttur.
- Pointer'lar esnekliği sağlamak ve dinamik depolama yönetimini kontrol etmek için kullanılır.

Copyright © 2015 Pearson. All rights reserved.